

Umbrella Notes — Module 4

Programming & Data Structures | shared core for Coal India MT (Systems), IBPS SO IT Officer (SPL-XVI), DSSSB TGT CS (DOE)

How to read this module

This is **full teaching notes** — learn directly from it. Depth is **balanced across all three exams** (middle depth by default: enough for IBPS SO, DSSSB TGT, and general PSU/Govt CS-IT-Systems exams).

■ **Green box** = how PSU/Govt exams actually test the topic (a reference lens, not a limit).

↑ **Amber box** = extra GATE-core depth needed *only* if targeting Coal India MT (or GATE-style PSUs). Skip for IBPS/DSSSB.

■ **Purple box** = a coverage warning (something kept light, or where an exam needs more/less).

Small sub-topics are folded under umbrella headings but never dropped. Topic-wise Q&A with explanations comes as a separate set after your walkthrough.

0 What this module covers

This module has two halves that exams treat as one area. **Programming** is the language machinery — data types, operators, control flow, functions, and the OOP concepts DSSSB and IBPS test. **Data structures** are the ways we organise data in memory so operations are efficient. The unifying idea: **choosing the right structure decides how fast your operations run**.

- **1. Programming fundamentals** — languages & translators, data types, operators, control flow, functions, recursion.
- **2. OOP concepts** — class/object, encapsulation, inheritance, polymorphism, abstraction (heavily tested).
- **3. Linear data structures** — arrays, linked lists, stacks, queues.
- **4. Non-linear data structures** — trees, binary search trees, heaps, graphs.
- **5. Hashing** — hash tables and collision handling.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

For IBPS SO IT and DSSSB, the heaviest-tested bits are **OOP concepts**, **stack/queue behaviour**, **linked-list vs array**, **tree terminology & traversals**, and **time complexity** of basic operations.

For programming, expect **C/output-prediction** and **operator/precedence** MCQs; DSSSB (DOE) adds C++/Java/OOP, NDMC adds Python.

1 Programming Fundamentals

1.1 Languages & translators

Source code must become machine code. Three translators do this:

Translator	Job	Note
Compiler	Translates the whole program at once	Reports all errors together; fast execution (C, C++)
Interpreter	Translates & runs line by line	Stops at first error; easier debugging (Python)
Assembler	Converts assembly language to machine code	One-to-one, low level

Languages sit on a spectrum: **low-level** (machine/assembly, hardware-specific) up to **high-level** (C, Java, Python — portable, human-readable). C is often called a **middle-level** language (high-level syntax with low-level access).

1.2 Data types, variables, operators

- **Primitive types:** integer, float/double, character, boolean. Size/range is a common MCQ (e.g. C int typically 2 or 4 bytes).
- **Variable** = named memory location; **constant** = fixed value. **l-value** (address, left of =) vs **r-value** (the value).
- **Operators:** arithmetic, relational, logical, bitwise, assignment, and the ternary **?:**. **Precedence & associativity** decide evaluation order — a frequent output-prediction trap.

1.3 Control flow

Three structures cover all logic: **sequence**, **selection** (if / if-else / switch), and **iteration** (for / while / do-while). **while** checks the condition first (may run zero times); **do-while** runs at least once. **break** exits a loop; **continue** skips to the next iteration.

1.4 Functions & recursion

- **Function** = reusable named block. **Parameters** (in the definition) vs **arguments** (passed in the call).
- **Pass by value** (a copy is sent; original unchanged) vs **pass by reference** (the address is sent; original can change).
- **Scope:** local (inside a function) vs global. **Recursion** = a function calling itself with a **base case** to stop; uses the call **stack** (deep recursion → stack overflow).

↑ Go deeper — GATE-core (Coal India MT)

Coal India / GATE go deeper into **C specifics**: pointers & pointer arithmetic, arrays-as-pointers, storage classes (auto/static/register/extern), memory layout (stack/heap), dynamic allocation (malloc/free), and recursion trace/output problems. IBPS/DSSSB need the concepts above, not deep pointer arithmetic.

2 Object-Oriented Programming (OOP)

OOP organises programs around **objects** (data + the operations on it) rather than around functions. It is one of the most-tested conceptual areas for IBPS SO IT and DSSSB (DOE lists C++, Java, and OOP explicitly).

2.1 Class vs object

A **class** is a blueprint/template (defines attributes + methods); an **object** is an instance of a class (an actual entity in memory). Analogy: class = the architectural plan, object = the built house.

2.2 The four pillars

Pillar	Meaning	Example / mechanism
Encapsulation	Bundling data + methods, hiding internal state behind access control	private/public; getters/setters
Abstraction	Exposing only essential features, hiding complexity	abstract classes, interfaces
Inheritance	A class derives properties from a parent class (code reuse)	'is-a' relation; base/derived
Polymorphism	One interface, many forms	overloading (compile-time), overriding (run-time)

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

The four pillars are a near-certain MCQ — memorise names + one-line meanings. Know examples: encapsulation via private members, inheritance = 'is-a', polymorphism via overloading vs overriding.

Overloading vs overriding is the classic trap: overloading = same name, different parameters, **compile-time**; overriding = redefining a parent method in a child, **run-time**.

2.3 Other terms exams touch

- **Constructor** (initialises an object, same name as class) vs **destructor** (cleans up).
- **Inheritance types**: single, multiple, multilevel, hierarchical, hybrid. (C++ allows multiple; Java uses interfaces instead — the 'diamond problem'.)
- **Access specifiers**: private (within class), protected (class + subclasses), public (everywhere).

3 Linear Data Structures

3.1 Arrays vs linked lists

	Array	Linked list
Memory	Contiguous block	Scattered nodes linked by pointers
Access	Random, $O(1)$ by index	Sequential, $O(n)$ to reach a node
Size	Fixed at declaration	Dynamic (grows/shrinks)
Insert/delete	Costly (shifting), $O(n)$	Cheap at a known node, $O(1)$

Linked-list variants: **singly** (one forward pointer), **doubly** (forward + backward), and **circular** (last node points to the first).

3.2 Stack (LIFO)

A **stack** is Last-In-First-Out — insert (**push**) and remove (**pop**) happen at the same end (**top**). Uses: function-call management (the call stack), **expression evaluation**, undo operations, and **recursion**. Applications exams love: converting **infix** → **postfix/prefix** and evaluating postfix.

3.3 Queue (FIFO)

A **queue** is First-In-First-Out — insert (**enqueue**) at the **rear**, remove (**dequeue**) from the **front**. Variants: **circular queue** (reuses freed space), **deque** (insert/delete at both ends), and **priority queue** (highest-priority element

served first). Uses: scheduling, buffering, BFS.

■ **Tested pattern & focus (PSU / Govt CS-IT-Systems)**

LIFO = stack, FIFO = queue is asked constantly. Know push/pop vs enqueue/dequeue terminology and which end each uses.

Application MCQs: recursion & function calls use a **stack**; CPU/print scheduling & BFS use a **queue**; **postfix conversion** uses a stack.

4 Non-Linear Data Structures

4.1 Tree terminology

A **tree** is a hierarchical structure of **nodes** with one **root** and no cycles. Key terms: **parent/child**, **leaf** (no children), **height** (longest root-to-leaf path), **depth** (distance from root), and **degree** (number of children). A **binary tree** allows at most two children per node.

4.2 Binary tree traversals

Traversal	Order	Note
Inorder	Left → Root → Right	On a BST, gives sorted order
Preorder	Root → Left → Right	Used to copy a tree
Postorder	Left → Right → Root	Used to delete a tree
Level-order	Level by level (BFS)	Uses a queue

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Traversal order questions are extremely common — memorise Inorder(LNR)/Preorder(NLR)/Postorder(LRN), and that inorder of a **BST** is sorted.

You may be given a tree and asked for its traversal sequence, or given traversals and asked to rebuild the tree.

4.3 Binary Search Tree (BST) & heaps

- **BST** — for every node, left subtree < node < right subtree. Search/insert/delete average $O(\log n)$, worst $O(n)$ (skewed tree).
- **Balanced trees** (AVL, Red-Black) keep height $\sim \log n$ so operations stay $O(\log n)$.
- **Heap** — a complete binary tree; **max-heap** (parent $\geq$ children) or **min-heap** (parent $\leq$ children). Root is the max/min. Used for **priority queues** and **heap sort**.

4.4 Graphs

A **graph** = vertices + edges; may be **directed/undirected** and **weighted/unweighted**. Represented by an **adjacency matrix** ($O(V^2)$ space, fast edge lookup) or **adjacency list** (space-efficient for sparse graphs). Traversals: **BFS** (uses a queue, explores level by level) and **DFS** (uses a stack/recursion, explores deep first).

↑ Go deeper — GATE-core (Coal India MT)

Coal India / GATE: traversal **output problems**, building expression trees, AVL rotations, heap build/heapify steps, and graph algorithms (covered fully in Module 6 — MST, shortest paths). Also height/node-count formulas for binary trees (max nodes at level $L = 2^L$).

5 Hashing

Hashing maps a key to an array index using a **hash function**, giving **average $O(1)$** search/insert/delete — the fastest lookup structure. When two keys map to the same index, that's a **collision**, resolved by:

- **Chaining** (open hashing) — each slot holds a linked list of colliding keys.
- **Open addressing** (closed hashing) — probe for the next free slot: **linear probing**, **quadratic probing**, or **double hashing**.

The **load factor** (entries $\div$ table size) governs performance; a high load factor means more collisions. A good hash function distributes keys uniformly.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Concept MCQs: definition of a **collision**, the resolution methods (chaining vs open addressing), and the **$O(1)$ average** lookup claim. Load-factor definition appears occasionally.

■ Coverage note / warning

Complexity notation is introduced lightly here and used throughout; its **formal treatment (Big-O, best/worst/average, complexity of each algorithm)** lives in **Module 6 (Algorithms)**. If a complexity question feels unfamiliar, that module is where it's taught in full.

Rapid recap — highest-yield facts

Area	Nail these
Translators	Compiler(whole, all errors) vs interpreter(line-by-line) vs assembler; C = middle-level
Programming	Primitive types; precedence/associativity; while(0+ times) vs do-while(1+); pass by value vs reference; recursion needs base case + uses stack
OOP	4 pillars: Encapsulation, Abstraction, Inheritance, Polymorphism; overloading(compile-time) vs overriding(run-time); class=blueprint, object=instance
Linear DS	Array(contiguous, $O(1)$ access, fixed) vs linked list(dynamic, $O(n)$ access); Stack=LIFO(push/pop); Queue=FIFO(enqueue/dequeue)
Trees	Inorder(LNR, sorted on BST)/Preorder(NLR)/Postorder(LRN)/Level-order(queue); BST $O(\log n)$ avg; heap = complete tree, max/min at root
Graphs	Adjacency matrix vs list; BFS(queue) vs DFS(stack/recursion)
Hashing	Hash function → index, $O(1)$ avg; collision → chaining or open addressing (linear/quadratic/double); load factor

✓ Before your walkthrough

That's the shared-core Programming & Data Structures picture at balanced depth. Flag any page/section that felt shaky and I'll go deeper; then the **topic-wise Q&A set** (real PSU/Govt questions + explanations, incl. output-prediction and traversal problems) follows for live practice.

Coal India target? The amber ↑ **GATE-core** boxes — pointers & memory, recursion tracing, AVL/heap operations, traversal output problems — are the extra depth this area needs. Algorithm complexity is taught fully in Module 6.